Música Solución

Academia de Música SonoRitmo — Solución propuesta en Java 21 + JavaFX 21 + Gson

Diseño de la solución

El modelo entregado (`ClaseMusical`, `ClaseIndividual`, `ClaseGrupal`, `ConSalaPrivada`) **no se modifica**. Sobre él se agregan capas, cada una con una sola responsabilidad:

Capa	Clases	Responsabilidad
Vista	`view/main-view.fxml`	Declara el formulario, la tabla, los botones y el área de mensajes. No contiene lógica.
Controlador	`controller.MainController`	Valida el formulario, arma el objeto del modelo, pide al repositorio guardar/eliminar y actualiza la interfaz.
Repository	`repository.ClaseMusicalRepository`	Mantiene la colección en memoria, guarda tras cada cambio y deshace si falla.
DAO	`dao.ClaseMusicalDao`, `dao.ClaseMusicalJsonDao`, `dao.PersistenciaException`	La interfaz declara las 4 operaciones que usa el repositorio; la clase las implementa leyendo y escribiendo JSON con Gson. No conoce JavaFX.

Cálculo de costo

Clase Individual: costo base de $60.000. Si el estudiante no rindió una evaluación de nivel, el costo se incrementa en un 15%.
Clase Grupal: costo base de $25.000. Si la cantidad de integrantes supera 6, el costo se incrementa en un 10%.
Ya lo resuelve el modelo: la tabla solo invoca `calcularCosto()`.

Cómo ejecutarla

Desde la carpeta del proyecto: `mvn javafx:run` (o ejecutar `Launcher` desde IntelliJ). Los archivos JSON se crean en la carpeta desde la que se ejecuta.

Cambios y archivos nuevos

`pom.xml`

Único cambio: la dependencia de Gson, en el lugar del comentario.

```
        <!-- Gson: lectura y escritura de archivos JSON -->
        <dependency>
            <groupId>com.google.code.gson</groupId>
            <artifactId>gson</artifactId>
            <version>2.11.0</version>
        </dependency>
```

`dao/ClaseMusicalDao.java`

Interfaz con solo las operaciones que el repositorio necesita: cargar y guardar cada subtipo. Se declara igual que `ConSalaPrivada`: solo la firma de los métodos.

```
package cl.duoc.sonoritmo.dao;

import java.util.List;
import cl.duoc.sonoritmo.model.ClaseIndividual;
import cl.duoc.sonoritmo.model.ClaseGrupal;

/**
 * Operaciones de lectura y escritura de clases que necesita el repositorio.
 * Cada subtipo se almacena por separado.
 */
public interface ClaseMusicalDao {

    List<ClaseIndividual> cargarClasesIndividuales();
    List<ClaseGrupal> cargarClasesGrupales();

    void guardarClasesIndividuales(List<ClaseIndividual> datos);
    void guardarClasesGrupales(List<ClaseGrupal> datos);
}
```

`dao/PersistenciaException.java`

Error propio, no comprobado, para fallas de lectura o escritura.

```
package cl.duoc.sonoritmo.dao;

/**
 * Error al leer o escribir el medio de almacenamiento.
 */
public class PersistenciaException extends RuntimeException {

    public PersistenciaException(String mensaje, Throwable causa) {
        super(mensaje, causa);
    }
}
```

`dao/ClaseMusicalJsonDao.java`

Clase que **implementa** `ClaseMusicalDao`. Para leer, Gson convierte el JSON a un arreglo del subtipo y se pasa a lista. Si el archivo no existe devuelve una lista vacía (primera ejecución sin errores).

```
package cl.duoc.sonoritmo.dao;

import cl.duoc.sonoritmo.model.ClaseIndividual;
import cl.duoc.sonoritmo.model.ClaseGrupal;
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.JsonParseException;

import java.io.IOException;
import java.io.Reader;
import java.io.Writer;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * Implementación de {@link ClaseMusicalDao} que guarda cada subtipo en un archivo JSON usando Gson.
 */
public class ClaseMusicalJsonDao implements ClaseMusicalDao {

    // Gson convierte objetos Java a JSON y al revés. setPrettyPrinting deja el archivo ordenado y legible.
    private final Gson gson = new GsonBuilder().setPrettyPrinting().create();
```

```
    // Los dos métodos cargar funcionan igual; solo cambian el archivo y el subtipo.
    @Override
    public List<ClaseIndividual> cargarClasesIndividuales() {
        Path archivo = Path.of("clases_individuales.json");
        // Primera ejecución: si el archivo aún no existe, se parte con una lista vacía (sin error).
        if (!Files.exists(archivo)) {
            return new ArrayList<>();
        }
        // try-with-resources: cierra el archivo automáticamente al terminar, haya error o no.
        try (Reader lector = Files.newBufferedReader(archivo, StandardCharsets.UTF_8)) {
            // Gson lee el JSON (una lista de objetos) como un arreglo del subtipo, usando sus
constructores vacíos.
            ClaseIndividual[] leidos = gson.fromJson(lector, ClaseIndividual[].class);
            // Si el archivo está vacío, fromJson devuelve null.
            if (leidos == null) {
                return new ArrayList<>();
            }
            // Arrays.asList devuelve una lista de tamaño fijo; se copia a un ArrayList para poder agregar
y quitar.
            return new ArrayList<>(Arrays.asList(leidos));
        // IOException: falla del archivo. JsonParseException: el contenido no es un JSON válido.
        // Se convierte en PersistenciaException para que el controlador muestre el mensaje.
        } catch (IOException | JsonParseException e) {
            throw new PersistenciaException("No se pudo leer el archivo clases_individuales.json.", e);
        }
    }

    @Override
    public void guardarClasesIndividuales(List<ClaseIndividual> datos) {
        escribir("clases_individuales.json", datos);
    }

    @Override
    public List<ClaseGrupal> cargarClasesGrupales() {
        Path archivo = Path.of("clases_grupales.json");
        if (!Files.exists(archivo)) {
            return new ArrayList<>();
        }
        try (Reader lector = Files.newBufferedReader(archivo, StandardCharsets.UTF_8)) {
            ClaseGrupal[] leidos = gson.fromJson(lector, ClaseGrupal[].class);
            if (leidos == null) {
                return new ArrayList<>();
            }
            return new ArrayList<>(Arrays.asList(leidos));
        } catch (IOException | JsonParseException e) {
            throw new PersistenciaException("No se pudo leer el archivo clases_grupales.json.", e);
        }
    }

    @Override
    public void guardarClasesGrupales(List<ClaseGrupal> datos) {
        escribir("clases_grupales.json", datos);
    }

    // Escribe la lista completa en el archivo (reemplaza lo que había). List<?> acepta una lista de
cualquier subtipo.
    private void escribir(String nombreArchivo, List<?> datos) {
        try (Writer escritor = Files.newBufferedWriter(Path.of(nombreArchivo), StandardCharsets.UTF_8)) {
            // toJson convierte la lista a texto JSON y lo escribe en el archivo.
            gson.toJson(datos, escritor);
        } catch (IOException | JsonParseException e) {
            throw new PersistenciaException("No se pudo guardar el archivo " + nombreArchivo + ".", e);
        }
    }
}
```

repository/ClaseMusicalRepository.java

Guarda una variable de tipo `ClaseMusicalJsonDao`. `agregar` y `eliminar` guardan al instante, de modo que memoria y archivos coinciden.

```
package cl.duoc.sonoritmo.repository;

import cl.duoc.sonoritmo.dao.ClaseMusicalJsonDao;
import cl.duoc.sonoritmo.model.ClaseMusical;
import cl.duoc.sonoritmo.model.ClaseIndividual;
import cl.duoc.sonoritmo.model.ClaseGrupal;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Centraliza las operaciones sobre la colección de clases y delega en el DAO
 * la lectura y escritura. Cada cambio se guarda de inmediato, de modo que la memoria
 * y los archivos siempre coinciden.
 */
public class ClaseMusicalRepository {

    private final ClaseMusicalJsonDao dao;
    private final List<ClaseMusical> clases = new ArrayList<>();

    public ClaseMusicalRepository(ClaseMusicalJsonDao dao) {
        this.dao = dao;
    }

    /**
     * Carga en memoria los datos guardados en los archivos.
     */
    public void cargar() {
        clases.clear();
        clases.addAll(dao.cargarClasesIndividuales());
        clases.addAll(dao.cargarClasesGrupales());
    }

    /**
     * @return vista de solo lectura de la colección
     */
    public List<ClaseMusical> listar() {
        // Vista de solo lectura: quien la reciba puede mirar la lista pero no modificarla; los cambios
pasan por agregar/eliminar.
        return Collections.unmodifiableList(clases);
    }

    /**
     * Agrega un elemento y guarda los archivos. Si no se puede guardar, deja la colección como estaba.
     */
    public void agregar(ClaseMusical nuevo) {
        clases.add(nuevo);
        try {
            guardar();
        // Si no se pudo guardar, se quita de la memoria lo recién agregado para que memoria y archivos
sigan iguales.
        } catch (RuntimeException e) {
            clases.remove(nuevo);
            // Se relanza el mismo error para que el controlador lo muestre en la ventana.
            throw e;
        }
    }

    /**
     * Elimina un elemento y guarda los archivos. Si no se puede guardar, deja la colección como estaba.
     */
    public void eliminar(ClaseMusical existente) {
        // indexOf devuelve -1 si el elemento no está en la lista; se guarda la posición para poder
deshacer.
        int posicion = clases.indexOf(existente);
        if (posicion < 0) {
            return;
        }
```

```
        clases.remove(posicion);
        try {
            guardar();
        } catch (RuntimeException e) {
            clases.add(posicion, existente);
            throw e;
        }
    }

    private void guardar() {
        List<ClaseIndividual> lista1 = new ArrayList<>();
        List<ClaseGrupal> lista2 = new ArrayList<>();
        // Se separan los elementos por subtipo porque cada subtipo se guarda en su propio archivo.
        for (ClaseMusical elemento : clases) {
            // "instanceof Tipo x" pregunta si el elemento es de ese subtipo y, si lo es, lo deja
disponible en x.
            if (elemento instanceof ClaseIndividual a) {
                lista1.add(a);
            } else if (elemento instanceof ClaseGrupal b) {
                lista2.add(b);
            }
        }
        dao.guardarClasesIndividuales(lista1);
        dao.guardarClasesGrupales(lista2);
    }
}
```

`view/main-view.fxml`

Vista declarada en FXML, vinculada al controlador con `fx:controller`. Cada control tiene su `fx:id`.

```
<?xml version="1.0" encoding="UTF-8"?>

<?import java.lang.String?>
<?import javafx.collections.FXCollections?>
<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.CheckBox?>
<?import javafx.scene.control.ComboBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TableColumn?>
<?import javafx.scene.control.TableView?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.ColumnConstraints?>
<?import javafx.scene.layout.GridPane?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.VBox?>

<VBox spacing="10.0" prefWidth="760.0" xmlns="http://javafx.com/javafx/21"
xmlns:fx="http://javafx.com/fxml/1" fx:controller="cl.duoc.sonoritmo.controller.MainController">
   <padding>
      <Insets bottom="15.0" left="15.0" right="15.0" top="15.0" />
   </padding>
   <Label text="SonoRitmo: registro de clases" />
   <GridPane hgap="10.0" vgap="8.0">
      <columnConstraints>
         <ColumnConstraints minWidth="150.0" />
         <ColumnConstraints hgrow="ALWAYS" minWidth="150.0" />
         <ColumnConstraints minWidth="150.0" />
         <ColumnConstraints hgrow="ALWAYS" minWidth="150.0" />
      </columnConstraints>
      <Label text="Tipo" GridPane.columnIndex="0" GridPane.rowIndex="0" />
      <ComboBox fx:id="cbTipo" prefWidth="260.0" promptText="Seleccione un tipo" GridPane.columnIndex="1"
GridPane.columnSpan="2" GridPane.rowIndex="0">
         <items>
            <FXCollections fx:factory="observableArrayList">
               <String fx:value="Clase Individual" />
               <String fx:value="Clase Grupal" />
            </FXCollections>
         </items>
      </ComboBox>
```

```
      <Label text="Código de clase" GridPane.columnIndex="0" GridPane.rowIndex="1" />
      <TextField fx:id="txtCodigoClase" GridPane.columnIndex="1" GridPane.rowIndex="1" />
      <Label text="Duración (meses)" GridPane.columnIndex="2" GridPane.rowIndex="1" />
      <TextField fx:id="txtDuracionMeses" GridPane.columnIndex="3" GridPane.rowIndex="1" />
      <Label text="Cupo máximo" GridPane.columnIndex="0" GridPane.rowIndex="2" />
      <TextField fx:id="txtCupoMaximo" GridPane.columnIndex="1" GridPane.rowIndex="2" />
      <Label text="Datos de la clase individual" GridPane.columnIndex="0" GridPane.rowIndex="3"
GridPane.columnSpan="4" />
      <Label text="Instrumento principal" GridPane.columnIndex="0" GridPane.rowIndex="4" />
      <TextField fx:id="txtInstrumento" GridPane.columnIndex="1" GridPane.rowIndex="4" />
      <CheckBox fx:id="chkEvaluacionNivel" mnemonicParsing="false" text="Rindió evaluación de nivel"
GridPane.columnIndex="2" GridPane.columnSpan="2" GridPane.rowIndex="4" />
      <CheckBox fx:id="chkSalaPrivada" mnemonicParsing="false" text="Sala de ensayo privada asignada"
GridPane.columnIndex="0" GridPane.columnSpan="2" GridPane.rowIndex="5" />
      <Label text="Datos de la clase grupal" GridPane.columnIndex="0" GridPane.rowIndex="6"
GridPane.columnSpan="4" />
      <Label text="Integrantes" GridPane.columnIndex="0" GridPane.rowIndex="7" />
      <TextField fx:id="txtIntegrantes" GridPane.columnIndex="1" GridPane.rowIndex="7" />
   </GridPane>
   <HBox spacing="10.0">
      <Button fx:id="btnRegistrar" mnemonicParsing="false" onAction="#onRegistrar" text="Registrar" />
      <Button fx:id="btnEliminar" mnemonicParsing="false" onAction="#onEliminar" text="Eliminar" />
   </HBox>
   <TableView fx:id="tabla" prefHeight="230.0">
      <columns>
         <TableColumn fx:id="colTipo" text="Tipo" />
         <TableColumn fx:id="colCodigoClase" text="Código" />
         <TableColumn fx:id="colDuracionMeses" text="Duración (meses)" />
         <TableColumn fx:id="colCupoMaximo" text="Cupo" />
         <TableColumn fx:id="colCosto" text="Costo" />
      </columns>
      <columnResizePolicy>
         <TableView fx:constant="CONSTRAINED_RESIZE_POLICY" />
      </columnResizePolicy>
   </TableView>
   <Label fx:id="lblMensaje" wrapText="true" minHeight="40.0" maxWidth="Infinity" />
</VBox>
```

`controller/MainController.java`

Controlador: `initialize()` configura columnas, carga datos y escucha el selector; `onRegistrar` y `onEliminar` son los manejadores de eventos.

```
package cl.duoc.sonoritmo.controller;

import cl.duoc.sonoritmo.dao.ClaseMusicalJsonDao;
import cl.duoc.sonoritmo.dao.PersistenciaException;
import cl.duoc.sonoritmo.model.ClaseMusical;
import cl.duoc.sonoritmo.model.ClaseIndividual;
import cl.duoc.sonoritmo.model.ClaseGrupal;
import cl.duoc.sonoritmo.repository.ClaseMusicalRepository;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.CheckBox;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.ListCell;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.TextField;

import java.text.DecimalFormat;
import java.text.DecimalFormatSymbols;
import java.util.Locale;


/**
 * Controlador de la vista principal: coordina el formulario, la tabla y los mensajes.
```

```
 * La lógica de negocio vive en el modelo y el acceso a datos en el repositorio.
 */
public class MainController {

    private static final String TIPO_CLASE_INDIVIDUAL = "Clase Individual";
    private static final String TIPO_CLASE_GRUPAL = "Clase Grupal";
    // Configuración regional de Chile: coma para los decimales y punto para los miles.
    private static final Locale CHILE = Locale.forLanguageTag("es-CL");
    // Formato de dinero: "$#,##0" muestra 21989 como $21.989.
    private static final DecimalFormat FORMATO_MONEDA = new DecimalFormat("$#,##0",
DecimalFormatSymbols.getInstance(CHILE));

    // Controles de la vista (main-view.fxml). Cada atributo @FXML debe llamarse igual que su fx:id.
    @FXML private ComboBox<String> cbTipo;
    @FXML private TextField txtCodigoClase;
    @FXML private TextField txtDuracionMeses;
    @FXML private TextField txtCupoMaximo;
    @FXML private TextField txtInstrumento;
    @FXML private CheckBox chkEvaluacionNivel;
    @FXML private CheckBox chkSalaPrivada;
    @FXML private TextField txtIntegrantes;
    @FXML private TableView<ClaseMusical> tabla;
    @FXML private TableColumn<ClaseMusical, String> colTipo;
    @FXML private TableColumn<ClaseMusical, String> colCodigoClase;
    @FXML private TableColumn<ClaseMusical, String> colDuracionMeses;
    @FXML private TableColumn<ClaseMusical, String> colCupoMaximo;
    @FXML private TableColumn<ClaseMusical, String> colCosto;
    @FXML private Label lblMensaje;

    // El repositorio guarda y carga los datos; el controlador solo le pide que lo haga.
    private final ClaseMusicalRepository repositorio = new ClaseMusicalRepository(new
ClaseMusicalJsonDao());
    // Lista observable: lo que se agregue o quite aquí se refleja de inmediato en la tabla.
    private final ObservableList<ClaseMusical> datosTabla = FXCollections.observableArrayList();

    @FXML
    public void initialize() {
        // Un ComboBox sin selección no muestra su texto de ayuda (promptText) por sí solo.
        // Esta celda personalizada escribe ese texto cuando no hay ningún elemento elegido.
        cbTipo.setButtonCell(new ListCell<>() {
            @Override
            protected void updateItem(String item, boolean vacio) {
                super.updateItem(item, vacio);
                setText(vacio || item == null ? cbTipo.getPromptText() : item);
            }
        });
        // Cada vez que el usuario cambia el tipo, se habilitan solo los campos de ese tipo.
        // (obs, anterior, nuevo) -> ... es una lambda: función corta que JavaFX ejecuta en cada cambio.
        cbTipo.valueProperty().addListener((obs, anterior, nuevo) -> actualizarCamposPorTipo());

        // setCellValueFactory le dice a cada columna qué texto mostrar en cada fila.
        // celda.getValue() es el objeto de esa fila; se envuelve en un SimpleStringProperty
        // porque la columna espera una "propiedad" y no un String suelto.
        // El tipo y el valor calculado no son atributos, por eso se piden con sus métodos del modelo.
        colTipo.setCellValueFactory(celda -> new SimpleStringProperty(celda.getValue().getTipo()));
        colCodigoClase.setCellValueFactory(celda -> new
SimpleStringProperty(celda.getValue().getCodigoClase()));
        colDuracionMeses.setCellValueFactory(celda -> new
SimpleStringProperty(String.valueOf(celda.getValue().getDuracionMeses())));
        colCupoMaximo.setCellValueFactory(celda -> new
SimpleStringProperty(String.valueOf(celda.getValue().getCupoMaximo())));
        colCosto.setCellValueFactory(celda -> new SimpleStringProperty(
                FORMATO_MONEDA.format(celda.getValue().calcularCosto())));
        // Enlaza la tabla con la lista observable: la tabla mostrará lo que tenga datosTabla.
        tabla.setItems(datosTabla);

        // Al abrir la ventana no hay tipo elegido, así que todos los campos específicos quedan
deshabilitados.
        // Luego se leen los archivos JSON; si falla la lectura, se avisa en la ventana y la app sigue.
        actualizarCamposPorTipo();
        try {
            repositorio.cargar();
```

```
        } catch (PersistenciaException e) {
            mostrarMensaje(e.getMessage());
        }
        refrescarTabla();
    }

    @FXML
    private void onRegistrar() {
        try {
            // Valida el formulario y crea el objeto; si algún dato es inválido lanza
IllegalArgumentException.
            ClaseMusical nuevo = construirDesdeFormulario();
            repositorio.agregar(nuevo);
            refrescarTabla();
            mostrarMensaje(nuevo.getCodigoClase() + " (" + nuevo.getTipo() + ") registrada
correctamente.");
            limpiarFormulario();
        // Un solo catch para dos tipos de error: dato inválido (formulario o modelo) y falla al guardar
archivos.
        // En ambos casos el mensaje del error se muestra en la ventana, no en la consola.
        } catch (IllegalArgumentException | PersistenciaException e) {
            mostrarMensaje(e.getMessage());
        }
    }

    @FXML
    private void onEliminar() {
        ClaseMusical seleccionado = tabla.getSelectionModel().getSelectedItem();
        // getSelectedItem() devuelve null cuando no hay ninguna fila seleccionada.
        if (seleccionado == null) {
            mostrarMensaje("Debe seleccionar una clase de la tabla para eliminarla.");
            return;
        }
        try {
            repositorio.eliminar(seleccionado);
            refrescarTabla();
            mostrarMensaje(seleccionado.getCodigoClase() + " (" + seleccionado.getTipo() + ") eliminada
correctamente.");
        } catch (PersistenciaException e) {
            mostrarMensaje(e.getMessage());
        }
    }

    /**
     * Valida el formulario y construye el objeto del tipo elegido. Lanza
     * IllegalArgumentException con el campo y el motivo si algún dato no es válido.
     */
    private ClaseMusical construirDesdeFormulario() {
        // getValue() devuelve null si no se eligió ningún tipo en el selector.
        String tipo = cbTipo.getValue();
        if (tipo == null) {
            throw new IllegalArgumentException("Tipo: debe seleccionar un tipo de clase.");
        }
        String codigoClase = leerTexto(txtCodigoClase, "Código de clase");
        int duracionMeses = leerEntero(txtDuracionMeses, "Duración (meses)", 1, 12, "debe estar entre 1 y
12.");
        int cupoMaximo = leerEntero(txtCupoMaximo, "Cupo máximo", 1, Integer.MAX_VALUE, "debe ser mayor que
cero.");

        // Según el tipo elegido se leen solo sus campos específicos y se crea el subtipo correspondiente.
        // Se compara con la constante a la izquierda para evitar errores si tipo fuera null.
        if (TIPO_CLASE_INDIVIDUAL.equals(tipo)) {
            String instrumento = leerTexto(txtInstrumento, "Instrumento principal");
            ClaseIndividual claseIndividual = new ClaseIndividual(codigoClase, duracionMeses, cupoMaximo,
instrumento, chkEvaluacionNivel.isSelected());
            // El constructor deja este dato en falso; si la casilla está marcada se activa con el método
de la interfaz.
            if (chkSalaPrivada.isSelected()) {
                claseIndividual.asignarSalaPrivada();
            }
            return claseIndividual;
        }
```

```
        // Segundo tipo: Clase Grupal
        int integrantes = leerEntero(txtIntegrantes, "Integrantes", 1, Integer.MAX_VALUE, "debe ser mayor
que cero.");
        ClaseGrupal claseGrupal = new ClaseGrupal(codigoClase, duracionMeses, cupoMaximo, integrantes);
        return claseGrupal;
    }

    private String leerTexto(TextField campo, String nombre) {
        String valor = campo.getText() == null ? "" : campo.getText().trim();
        if (valor.isEmpty()) {
            throw new IllegalArgumentException(nombre + ": no puede estar vacío ni contener solo
espacios.");
        }
        return valor;
    }

    private int leerEntero(TextField campo, String nombre, int minimo, int maximo, String regla) {
        int valor;
        try {
            // parseInt lanza NumberFormatException si el texto no es un número entero (por ejemplo "abc" o
"2.5").
            valor = Integer.parseInt(campo.getText().trim());
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException(nombre + ": debe ser un número entero.");
        }
        // Integer.MAX_VALUE como máximo significa "sin límite superior".
        if (valor < minimo || valor > maximo) {
            throw new IllegalArgumentException(nombre + ": " + regla);
        }
        return valor;
    }

    private double leerDecimalPositivo(TextField campo, String nombre) {
        double valor;
        try {
            // Se cambia la coma por punto para aceptar "22,5" y "22.5"; parseDouble solo entiende el
punto.
            valor = Double.parseDouble(campo.getText().trim().replace(',', '.'));
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException(nombre + ": debe ser un número (se aceptan decimales).");
        }
        if (valor <= 0) {
            throw new IllegalArgumentException(nombre + ": debe ser mayor que cero.");
        }
        return valor;
    }

    /** Habilita solo los datos propios del tipo elegido; los demás se deshabilitan y se vacían. */
    private void actualizarCamposPorTipo() {
        String tipo = cbTipo.getValue();
        boolean esClaseIndividual = TIPO_CLASE_INDIVIDUAL.equals(tipo);
        boolean esClaseGrupal = TIPO_CLASE_GRUPAL.equals(tipo);
        // setDisable(true) bloquea el control. Se deshabilita todo campo que NO pertenece al tipo elegido,
        // y se vacía para que no quede un valor viejo que se confunda al registrar.
        txtInstrumento.setDisable(!esClaseIndividual);
        if (!esClaseIndividual) {
            txtInstrumento.clear();
        }
        chkEvaluacionNivel.setDisable(!esClaseIndividual);
        if (!esClaseIndividual) {
            chkEvaluacionNivel.setSelected(false);
        }
        chkSalaPrivada.setDisable(!esClaseIndividual);
        if (!esClaseIndividual) {
            chkSalaPrivada.setSelected(false);
        }
        txtIntegrantes.setDisable(!esClaseGrupal);
        if (!esClaseGrupal) {
            txtIntegrantes.clear();
        }
    }
```

```
    private void limpiarFormulario() {
        txtCodigoClase.clear();
        txtDuracionMeses.clear();
        txtCupoMaximo.clear();
        // null deja el selector sin elección (vuelve a verse el texto de ayuda).
        cbTipo.setValue(null);
        actualizarCamposPorTipo();
    }

    private void refrescarTabla() {
        // setAll reemplaza todo el contenido de la lista observable; la tabla se redibuja sola.
        datosTabla.setAll(repositorio.listar());
    }

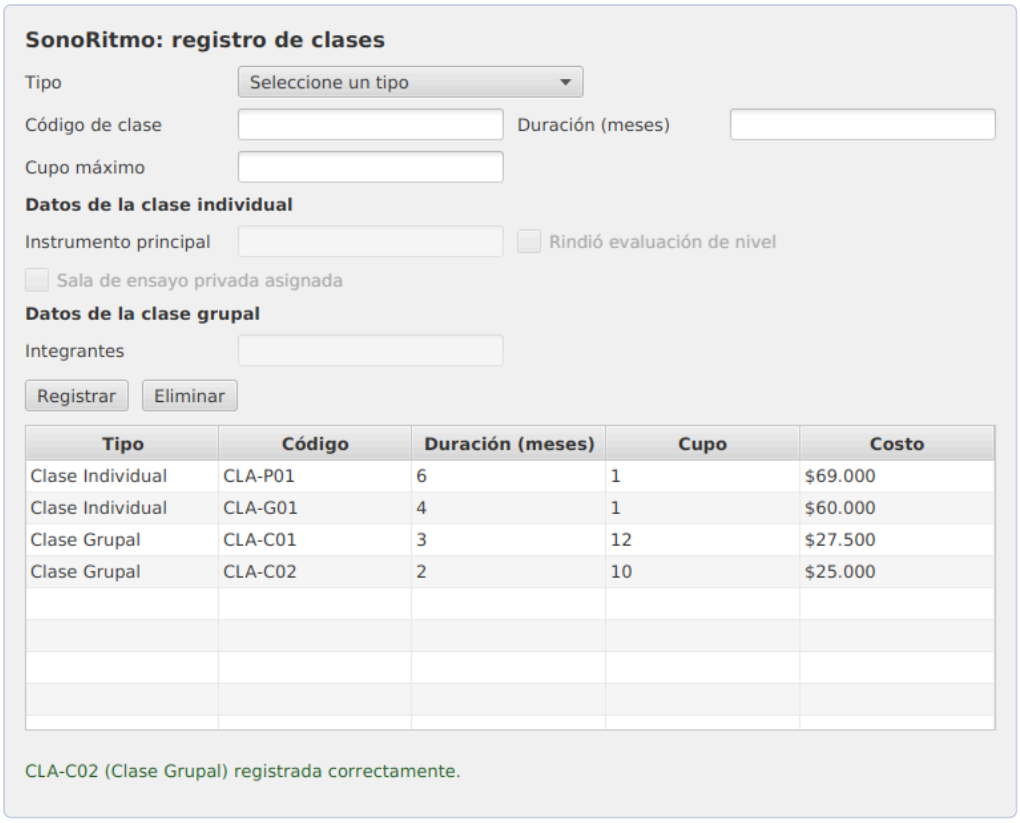
    private void mostrarMensaje(String mensaje) {
        lblMensaje.setText(mensaje);
    }
}
```

Comprobación realizada

La solución se compiló y ejecutó con los datos de la guía; el resultado de cada paso coincide con lo esperado:

Tipo	Código de clase	Duración (meses)	Cupo máximo	Costo calculado
Clase Individual	CLA-P01	6	1	$69.000
Clase Individual	CLA-G01	4	1	$60.000
Clase Grupal	CLA-C01	3	12	$27.500
Clase Grupal	CLA-C02	2	10	$25.000

- Eliminar sin selección → aviso en la ventana y sin cambios.
- Duración (meses) = 15 → el mensaje nombra el campo y se conservan los valores.
- Eliminar CLA-P01, cerrar y reabrir → la tabla muestra solo los tres restantes; los JSON coinciden con la tabla.
- Sin archivos JSON → la aplicación inicia con la tabla vacía y sin mensajes.



Ventana de la solución con los cuatro registros.

Solución propuesta como material de estudio. Existen otras formas válidas de resolver el caso.